

RUI TANG (SCOTT)

Ruit@usc.edu | 747-222-0001 | [GitHub Profile](#) | [LinkedIn Profile](#)

EDUCATION

University of Southern California, Viterbi School of Engineering

Aug. 2022 - May. 2026 (Expected)

Bachelor of Science in Computer Science (Games)

Los Angeles, USA

Relevant Coursework: Data Structures & Algorithms, Operating Systems, Distributed Systems, Computer Networks, Applications of Machine Learning, Game Engines Development, Video Game Programming, Game Design & Production, Software Engineering Principles

TECHNICAL SKILLS

Programming Languages: C++, C#, Java, SQL, HLSL, Python, Lua

Game Engines & Tools: Unity, Unreal Engine 5, SDL Library, DirectX 11, CUDA

Programming Concepts: Multi-threading, Networking, SDLC, Shader Programming, Memory Management

Working Languages: English(fluent), Mandarin(fluent)

WORK EXPERIENCES

ByteDance | Douyin - TikTok Parent Company

May. 2025 - Aug. 2025

Software Engineer Intern - 3D Engine Development

Beijing, China

- Implemented and shipped modules for Douyin interactive features (e.g., co-parenting pets) reaching 10M+ DAU and supported early prototyping for upcoming AIGC-driven UGC gameplay systems.
- Architected a modular asset pipeline in Unity, integrating with ByteDance's SAR engine, using a factory method pattern with staged processing (pre/post workflows). Built extensible modules and custom Unity Editor tools to transform meshes, shaders, and materials into deployable SAR bundles through a TypeScript interface toolchain, enabling cross-engine publishing and reducing manual conversion overhead.
- Implemented a custom Dynamic Bone runtime system mapping JSON-defined colliders to skeleton nodes and constructing recursive particle chains; optimized simulation stability and performance, fully replacing the previous solution across platforms.
- Optimized client performance for group chat and low-end devices through LOD replacement and frame cache mechanisms; resolved 20+ shader parameter inconsistencies between Unity and SAR to improve rendering stability and visual consistency.

NetEase Games | Largest game development companies in China

May. 2024 - Aug. 2024

Game Designer Intern

Hangzhou, China

- Designed and iterated over 10+ interactive levels for Eggy Party using NeoX engine's internal editor and scripting tools, focusing on obstacle placement, level progression, difficulty tuning, and UGC template design to drive player engagement across varying skill levels.
- Collaborated with design leads, engineers, and artists to refine gameplay mechanics and flow; led structured playtests and adjusted level layouts and parameters to improve onboarding experience, game balance, and win-rate consistency.
- Analyzed heatmaps and player behavior data via internal dashboards to identify failure hotspots and early churn; refined trap timing and reward pacing, contributing to an ~8% increase in Day 3 retention during live seasonal content rollout.

iFlytek | Leading AI technology company in China

May. 2023 - Aug. 2023

Software Development Engineer Intern

Shanghai, China

- Integrated real-time ASR and NLP models into iFlytek's intelligent simultaneous interpretation system by implementing model invocation interfaces and aligning time-stamped text streams with audio input in C++.
- Utilized Valgrind to trace and fix memory management issues in the live captioning module, improving system stability for long translation sessions.
- Assisted in optimizing the ASR decoding pipeline and downstream NLP integration, significantly reducing average processing delay by ~150ms in test environments and enhancing real-time performance and system stability across the live captioning and multilingual translation platform.

SELECTED PROJECTS

Lucky Duckies (Unreal Engine Co-op Game) | Incoming

Aug. 2025 - Current

Systems & UI Engineer

- Shipped gameplay systems and fixed production issues in a 100,000+ LOC Unreal Engine codebase within a 40+ member cross-disciplinary team using sprint-based development
- Led system engineering for the dialogue framework built on Unreal's replication architecture, implementing dialogue state management, server-authoritative logic, and RPC synchronization to ensure consistent NPC interactions across clients.
- Implemented HUD and world-space UI (tutorials, progression, boss health, prompts), ensuring responsive updates under networked environment.
- Engineered immersive cutscene flow and a hybrid online/local character selection level for co-op split-screen experience, enabling multiplayer state transitions and seamless input hot-swapping (controller/mouse)

Slash Bash (Unity Action Fighting Game) | [itch.io Link](#)

Jul. 2024 - Jan. 2025

Lead Gameplay Programmer & Systems Designer

- Developed combo-based melee battle system using features like Animator Controller and ScriptableObjects, with a circular hit detection for simple/combo attacks, custom damage/block logic, and character-specific abilities, with 95% of playtesters rating combat as "fluid and satisfying."
- Built a modular, event-driven FSM managing player states (idle, move, attack, block), using animation events and input buffering to trigger transitions; movement and knockback handled via Rigidbody2D velocity manipulation. Ensuring smooth animations and responsive controls.
- Implemented local 2-player combat using Unity's Input System, with nine characters, four interactable maps, player-specific input mapping, and camera tracking. Refined input handling to prevent control overlap and improve responsiveness, reducing input misreads by 20% during playtesting.
- Published a demo on itch.io with 100+ downloads in the first week and a 4.5/5 average rating based on player feedback on animations and variety.

DirectX11 Custom Game Engine | [GitHub Link](#)

Feb. 2025 - April. 2025

Engine Programmer

- Built a multi-threaded rendering engine in C++ with DirectX11, featuring a delta-time game loop, resource management, and scene graph updates, and supporting skeletal animation with skinned mesh shaders, GPU-driven skinning, and inverse kinematics.
- Implemented real-time graphics features including Phong and Half-Lambert lighting, normal mapping, and multi-pass post-processing (bloom, tone mapping) within a custom rendering pipeline.
- Optimized performance with SIMD operations, custom memory allocators (boundary tag allocator), and cache-aware data layouts, and integrated profiling/debugging tools to detect memory leaks and GPU bottlenecks.

SDL2 Game Projects | [GitHub Link](#)

Aug. 2024 - Dec. 2024

Engine & Gameplay Programmer

- Built a custom 2D game engine in C++ using SDL2, implementing delta-time-based game loop updates, input/output systems, foundational math modules (vector operations, Lerp, quaternion rotation), and AABB collision detection. Integrated a stack allocator to improve memory efficiency.
- Implemented A* pathfinding and a node-based navigation system (BFS-driven) for enemy AI behavior across multiple engine-driven games. Optimized movement strategies using node weighting and directional bias to enhance path efficiency in complex tile-based environments.